

PONTIFICIA UNIVERSIDAD CATÓLICA DE CHILE
ESCUELA DE INGENIERÍA
DEPARTAMENTO DE CIENCIA DE LA COMPUTACIÓN

Tópicos Avanzados en Teoría de la Computación - IIC3810
Tarea 2
Fecha de entrega: jueves 8 de octubre

Respuestas

Pregunta 3

Ejecución en IBM Quantum

El programa `pregunta3.py` se ejecutó en un computador cuántico real de IBM. Para cada función se transpiló el circuito `circuito_deutsch_jozsa(oraculo, 7)` para el computador elegido (con `generate_preset_pass_manager` y `optimization_level=3`) y se envió un trabajo con 1000 ejecuciones (*shots*) usando la primitiva `SamplerV2` de `qiskit-ibm-runtime`.

Computador cuántico	`ibm_fez` (procesador Heron r2, 156 qubits)
Fecha de ejecución	11 de octubre de 2026, 21:16 (hora de Chile; 12/10/2026 00:16 UTC)
Job ID para f_1	`db62dn2mb58s738b5r4g`
Job ID para f_2	`db62dnclf4us73c4na0g`
Ejecuciones por función	1000

Cuadro 1: Datos de la ejecución.

Circuitos transpilados

	Compuertas de dos qubits	Profundidad
f_1	112	240
f_2	0	7

Cuadro 2: Tamaño de los circuitos de Deutsch-Jozsa después de transpilar para `ibm_fez`.

El circuito transpilado para f_1 tiene **112 compuertas de dos qubits** y **profundidad 240**, bastante menos que la referencia de alrededor de 500 compuertas de dos qubits indicada en el enunciado. Esto se debe a cómo se construyó el oráculo en `pregunta2.py`: usando que $f_1(x) = 1 \oplus \mathrm{MAJ}_7(x)$, se calcula $|x|_1$ con tres sumadores completos ($|x|_1 = s + 2(c_1 + c_2 + c_3)$), de modo que $|x|_1 \geq 4$ si y solo si $\mathrm{MAJ}_3(c_1, c_2, c_3) = 1$.

Esto requiere solo 3 qubits auxiliares, y como la parte de cómputo se deshace con su inversa, se pueden usar compuertas Toffoli de fase relativa (3 CNOT en vez de 6). El oráculo de $f_2 = 0$ es la identidad, por lo que su circuito solo tiene compuertas de un qubit.

Resultados

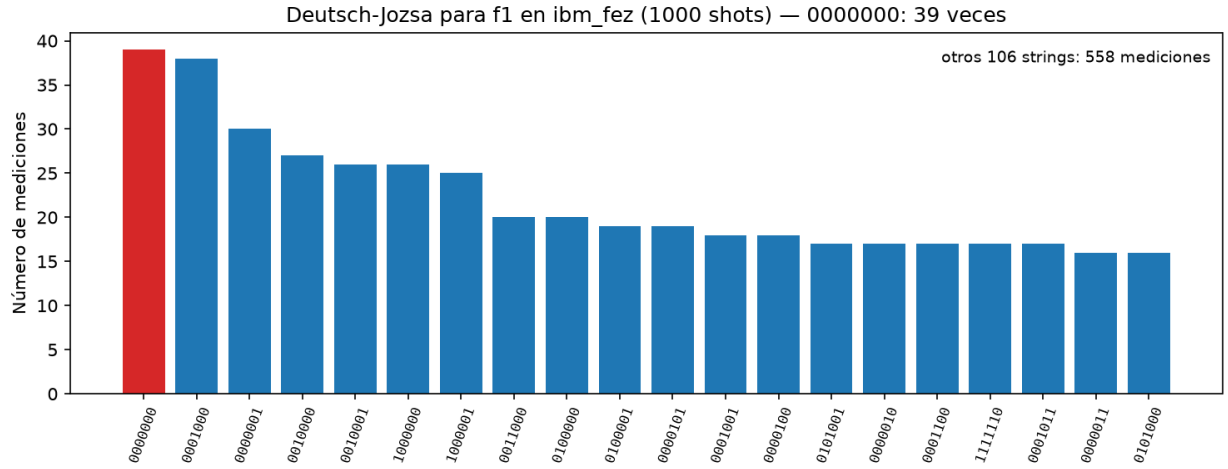


Figura 1: Resultados para f_1 en `ibm_fez` (20 strings más frecuentes; las 558 mediciones restantes se reparten entre otros 106 strings). El string 0^7 se obtuvo **39 veces** de 1000.

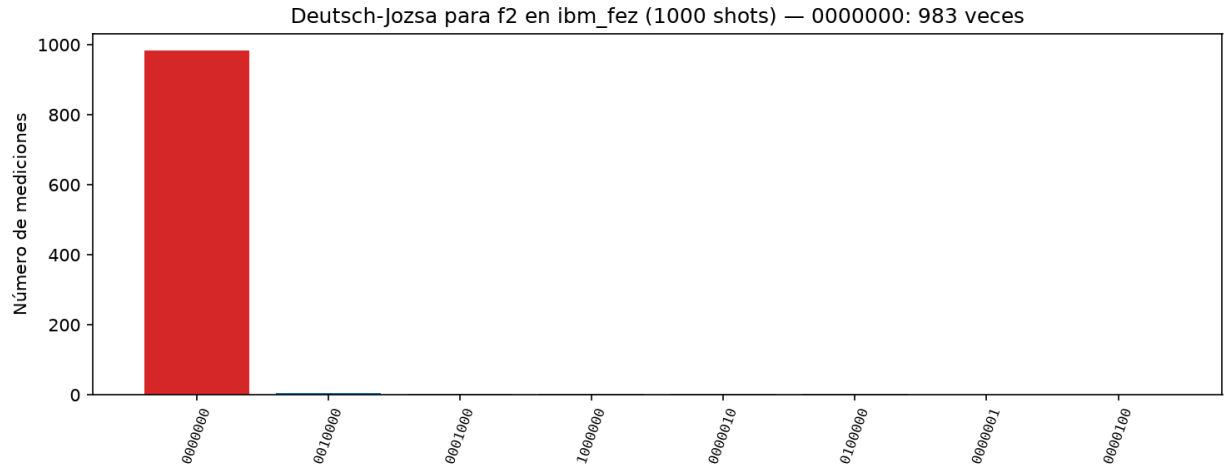


Figura 2: Resultados para f_2 en `ibm_fez` (todos los strings obtenidos). El string 0^7 se obtuvo **983 veces** de 1000.

	Veces que se obtuvo 0^7	Fracción	Strings distintos obtenidos
f_1	39	3,9%	126
f_2	983	98,3%	8

Cuadro 3: Resumen de los resultados.

Criterio utilizado

En el algoritmo de Deutsch-Jozsa, la probabilidad de medir 0^n es

$$P(0^n) = \left| \frac{1}{2^n} \sum_{x \in \{0,1\}^n} (-1)^{f(x)} \right|^2 ,$$

que vale 1 si f es constante y 0 si f es balanceada. Por esto, en un simulador ideal basta con revisar si *todas* las mediciones entregan 0^n, que es el criterio de la función `is_constant`.

En un computador real este criterio no es adecuado, porque las compuertas y las mediciones tienen errores. Por ejemplo, para f_2 se obtuvieron 17 resultados distintos de 0^7 (debidos a errores de las compuertas de un qubit y de lectura), por lo que `is_constant` concluiría erróneamente que f_2 es balanceada. Del mismo modo, el ruido hace que en f_1 aparezca 0^7 aunque idealmente tiene probabilidad 0. En el caso extremo de ruido total, la salida sería uniforme y $P(0^7) = 1/128 \approx 0{,}8\%$.

Por esto, en `pregunta3.py` se usa un criterio distinto, basado en la **fracción** de mediciones que entregan 0^7:

$$f \text{ es constante} \iff \frac{\#\{\text{mediciones iguales a } 0^7\}}{1000} \geq 0{,}5.$$

El umbral 0,5 es el punto medio entre los valores ideales 0 (balanceada) y 1 (constante), por lo que tolera que el ruido desplace la fracción en casi la mitad hacia cualquiera de los dos lados.

Justificación con los resultados obtenidos. Para f_1 la fracción de 0^7 fue 3,9%, muy por debajo de 0,5, por lo que se concluye correctamente que f_1 es balanceada. Para f_2 la fracción fue 98,3%, muy por encima de 0,5, por lo que se concluye correctamente que f_2 es constante. Ambos resultados están lejos del umbral: con 1000 ejecuciones, la desviación estándar de la fracción estimada es a lo más $\sqrt{0{,}25/1000} \approx 0{,}016$, y las fracciones obtenidas están a unas 29 (f_1) y 30 (f_2) desviaciones estándar de 0,5. Por lo tanto, la decisión no depende de fluctuaciones estadísticas. La diferencia entre ambos casos es coherente con el tamaño de los circuitos: el de f_2 no tiene compuertas de dos qubits y casi no acumula ruido, mientras que el de f_1 tiene 112, lo que distribuye los resultados entre 126 strings distintos, pero sin acercar la fracción de 0^7 al umbral.

Resultado de ejecutar `pregunta3.py`

```
> python pregunta3.py
Computador: ibm_fez
f1: circuito transpilado con 112 compuertas de dos qubits, profundidad 240
f1: job ID = db62dn2mb58s738b5r4g
f2: circuito transpilado con 0 compuertas de dos qubits, profundidad 7
f2: job ID = db62dnclf4us73c4na0g
f1: 0000000 obtenido 39/1000 veces (3.9%)
f2: 0000000 obtenido 983/1000 veces (98.3%)

f1 es balanceada
f2 es constante
```